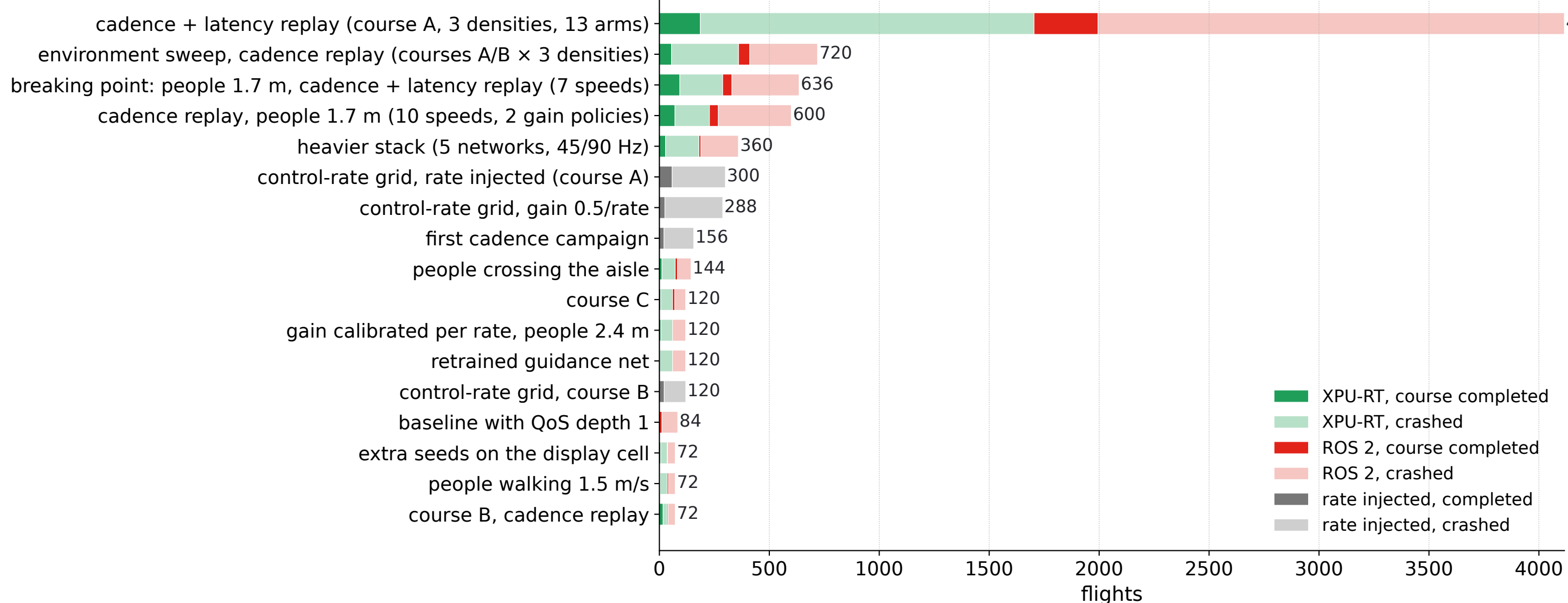
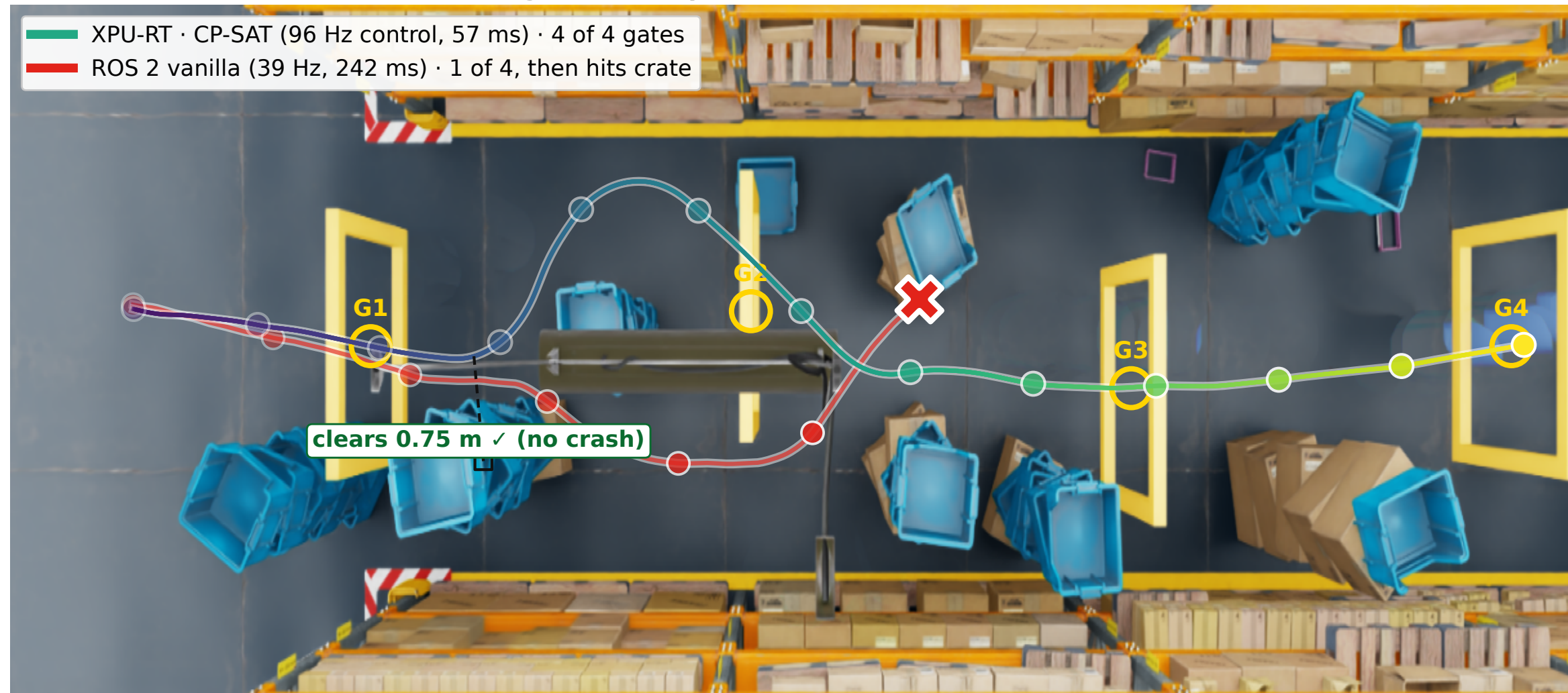


One workload, two runtimes, one board — the same kernels scheduled by XPU-RT's solvers against ROS 2 as one writes it: what was run, what the K1 delivered, where the flights ended

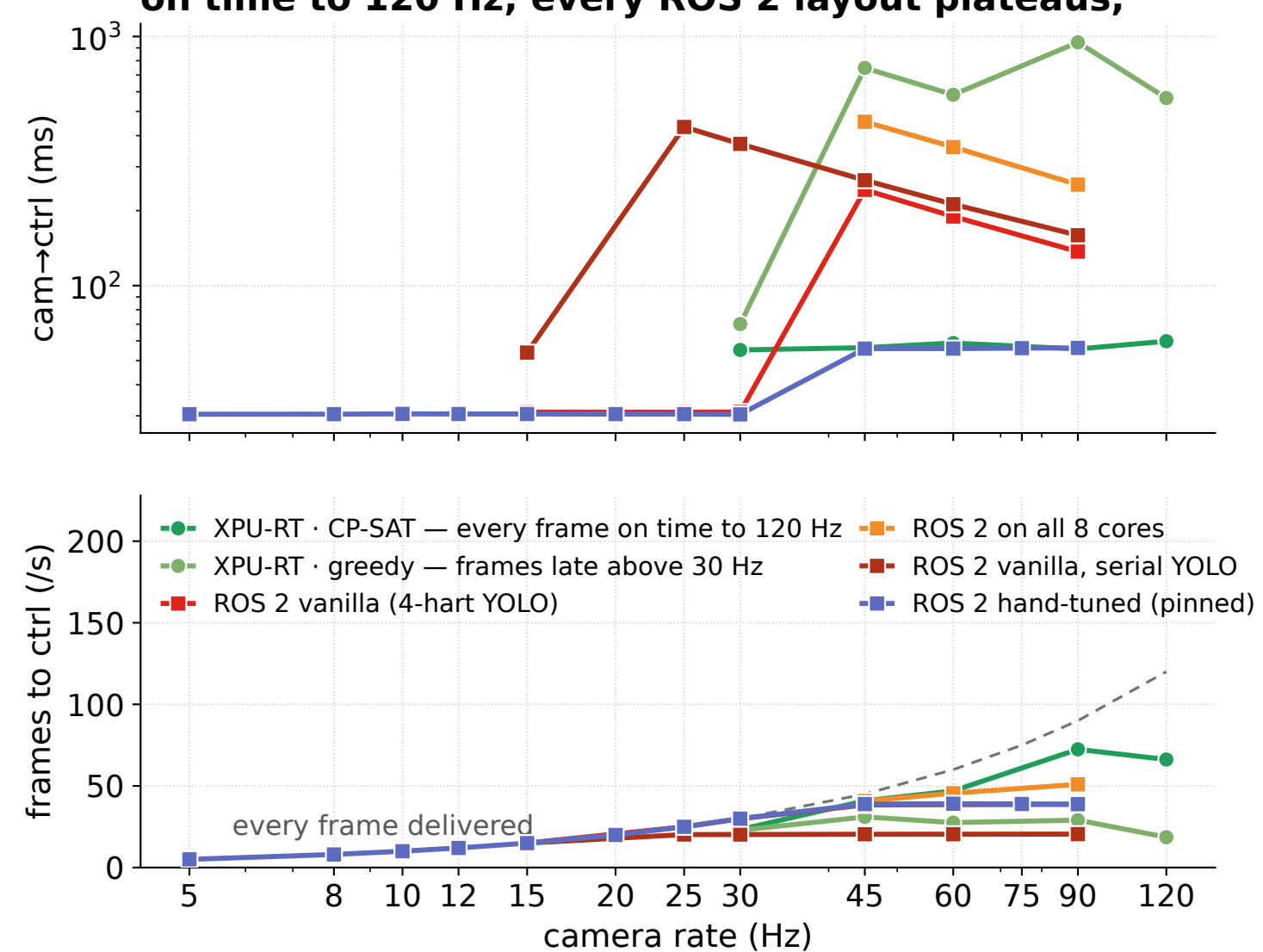
A What was run: 8,100 recorded flights in 17 campaigns (12 seeds a cell), 319 ROS 2 and 249 XPU-RT runs on the K1, 64 solved tables executed, 6 feedback studies



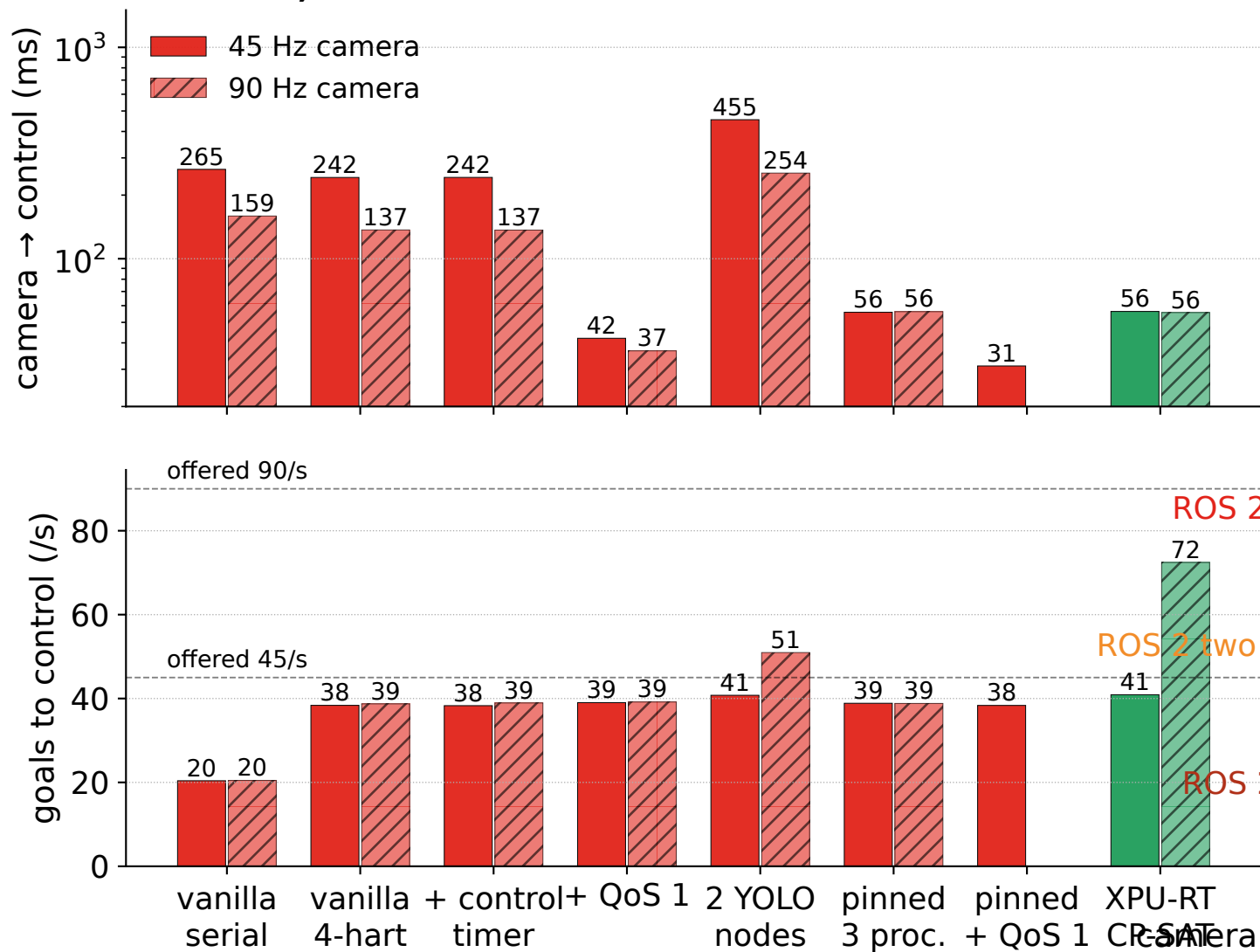
B The same scene, controller and gain — only what the K1 delivers differs (1.0 m/s, people 2.4 m)



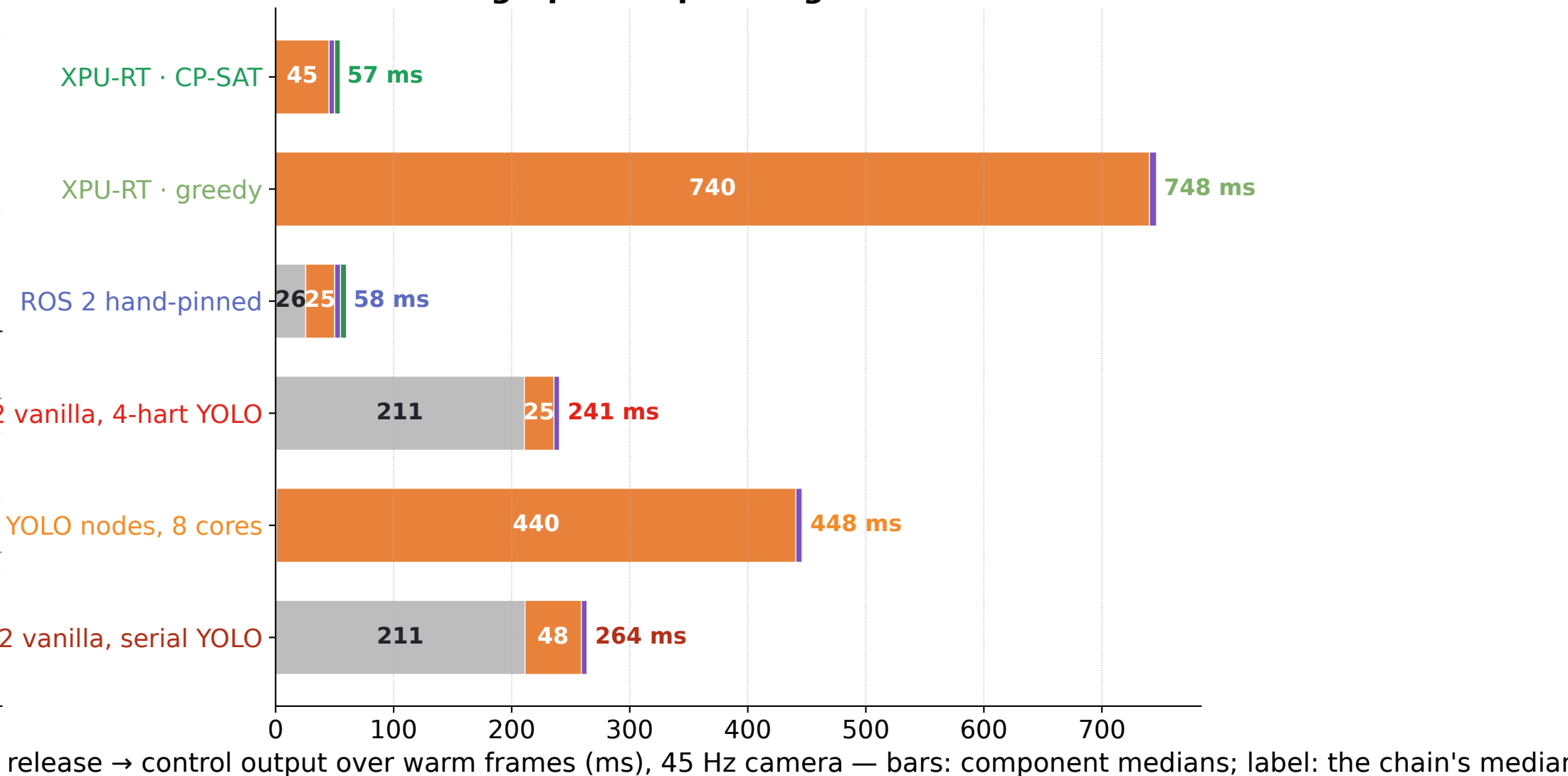
C frequency on the K1: the solved table keeps every frame on time to 120 Hz; every ROS 2 layout plateaus,



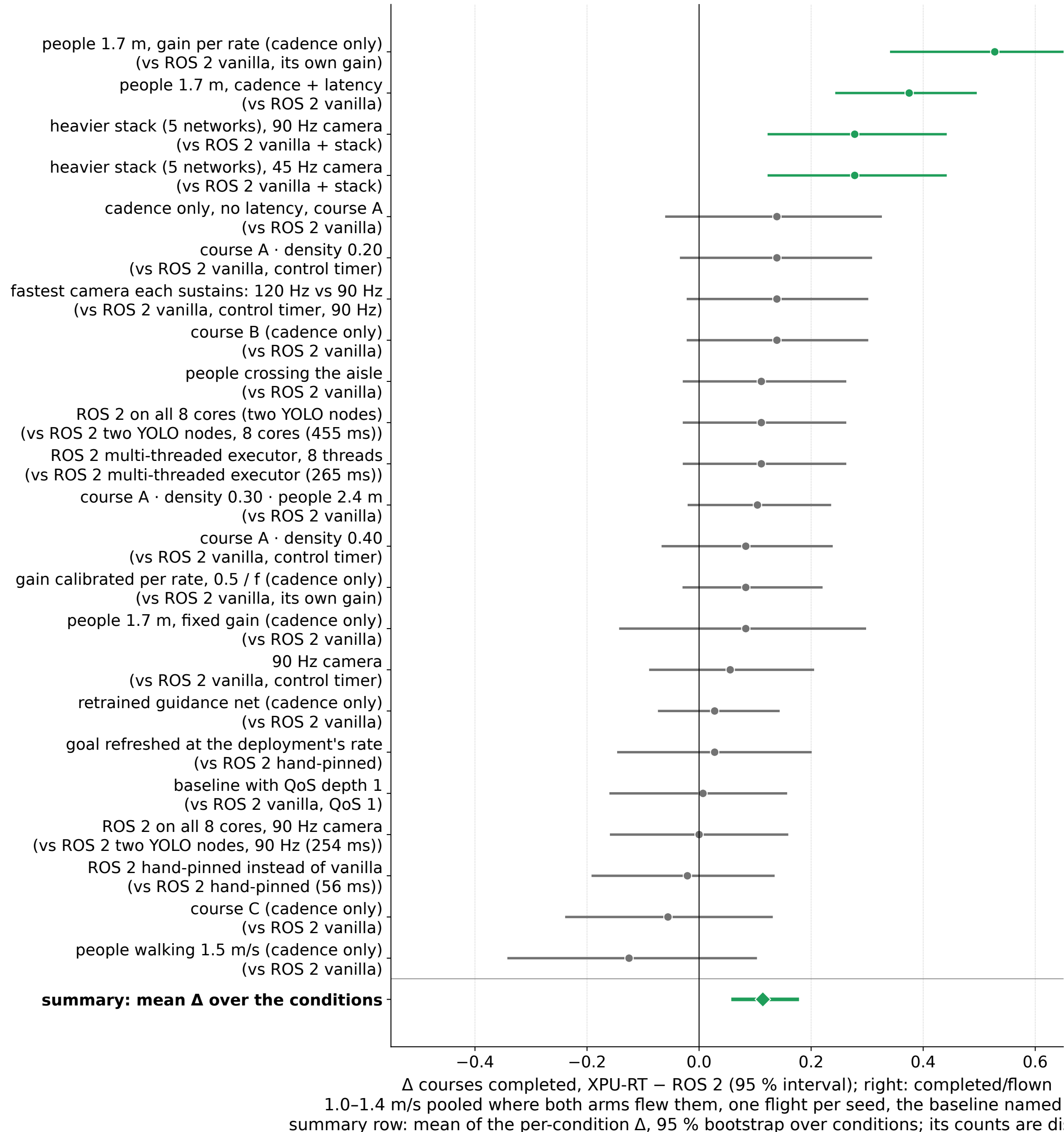
D Hand-tuning ROS 2 on the K1: pinning reaches the schedule's latency at 45 Hz, but every layout stops at 20-51 goals/s at 90 Hz, where the solved table delivers 72



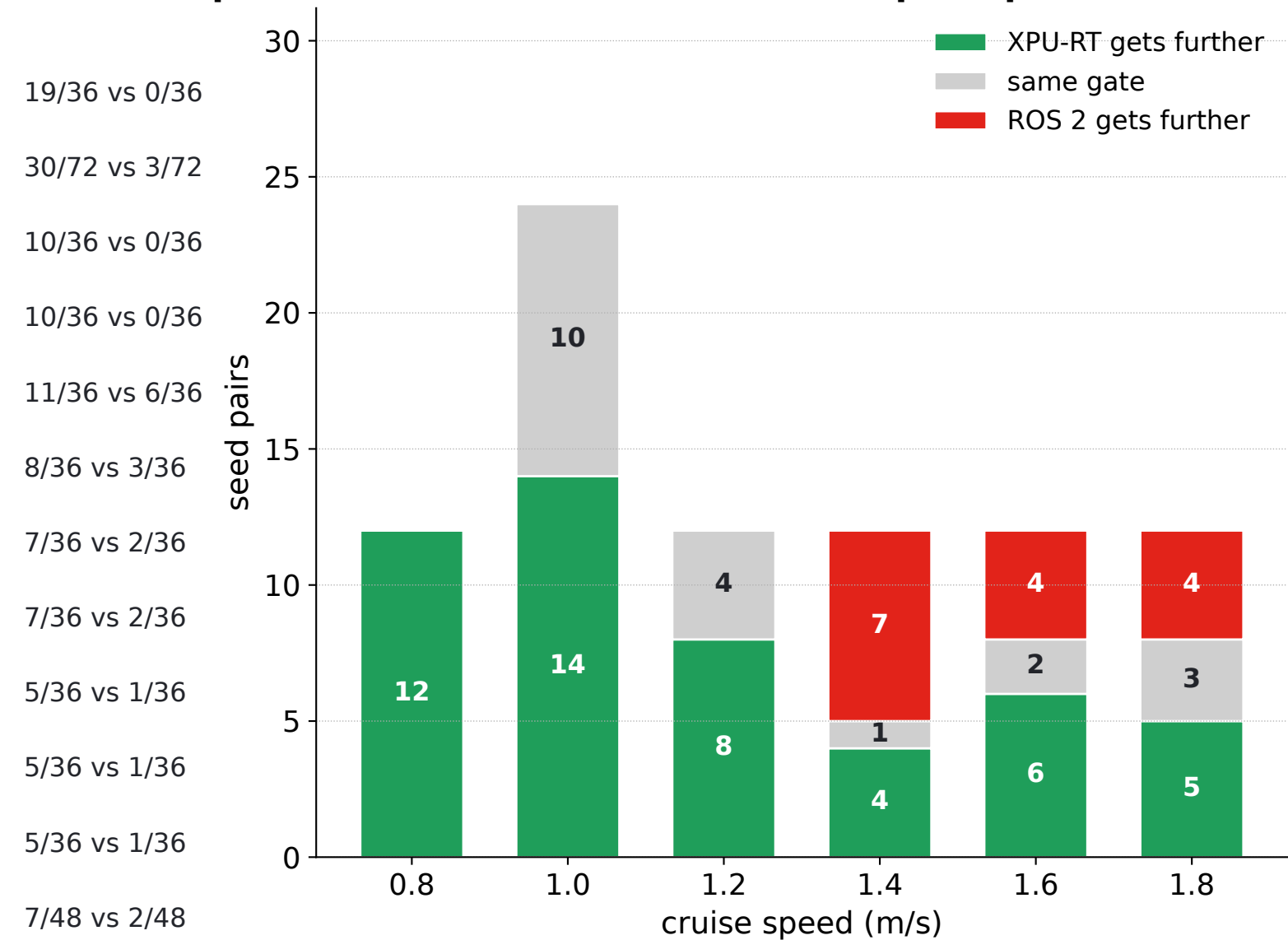
E Where the latency goes: the schedule's chain is compute; the callback graph's is queueing



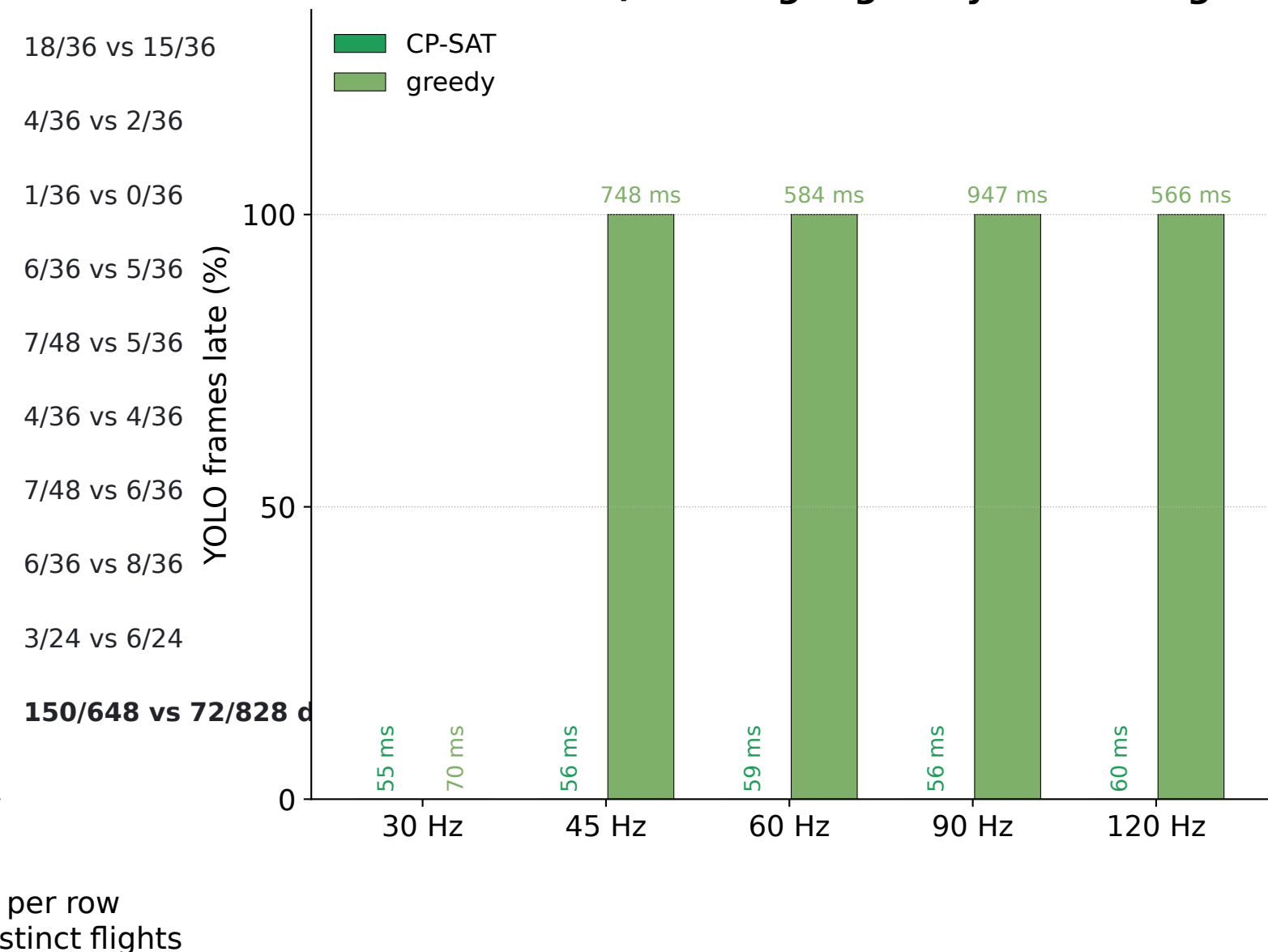
F Every condition flown by both arms: XPU-RT ahead in 19 of 23 (clear of zero in 4, behind in 0); mean Δ = +0.11 [+0.06, +0.18], bootstrap over 60 trials



G Same seed, both runtimes: XPU-RT further on 49, ROS 2 on 15, tie on 20 seed-speed pairs



H Same spec, two solvers: CP-SAT holds every window at every camera rate; greedy loses frames above 30 Hz (labels: camera→control) — in flight greedy clears no gate



I Added load: two more networks on the same eight harts — CP-SAT > greedy > ROS 2 on the board and in flight

